

FORMATION EMBARQUÉE

TP5 — Seance 2

Formation Systèmes embarqués & programmation bas niveau

Systèmes embarqués · bas niveau · automatismes

TP 5 — Fiche de séance 2 : driver BME280 écrit maison (3 h)

En-tête pédagogique

Objectifs	Lire un datasheet ; écrire un driver I2C sans bibliothèque ; arithmétique en point fixe ; gérer les erreurs bus sans bloquer
Prérequis	Séance 1 ; module 10 §4.5 (I2C HAL) ; module 01 §2.1 (point fixe)
Outils	CubeIDE ; capteur BME280 (ou potentiomètre en substitut) ; datasheet Bosch BME280 ouvert
Livrable	<code>drivers/bme280.{c,h}</code> retournant T/P/H plausibles, erreurs gérées

Interdiction d'utiliser une bibliothèque toute faite. Le but EST d'écrire le driver. C'est ce qui te sépare d'un « assembleur de bibliothèques » et fait de toi un développeur firmware.

Déroulé minuté

0:00-0:30 — Configurer I2C et scanner le bus

1. CubeMX : activer I2C1 (PB8/PB9 sur Nucleo), 100 kHz (Standard Mode). Régénérer.
2. **Scanner I2C** (le « hello world » du bus) :

```
for (uint8_t a = 1; a < 128; a++)
    if (HAL_I2C_IsDeviceReady(&hi2c1, a << 1, 1, 5) == HAL_OK)
        printf("trouve : 0x%02X\r\n", a);
```

Attendu : `0x76` ou `0x77` . ⚠ **Piège n°1** : la HAL attend l'adresse **décalée d'un bit à gauche** (`a << 1`), contrairement à Wire d'Arduino.

1. Lire le registre `id` (0xD0) : doit répondre **0x60** (l'identifiant du BME280). C'est ta première transaction réussie.

0:30-1:00 — L'interface du driver

`drivers/bme280.h` — l'interface imposée (pense « ce que l'appelant a le droit de savoir ») :

```

#ifndef BME280_H
#define BME280_H
#include "stm32f4xx_hal.h"
#include <stdbool.h>
#include <stdint.h>

typedef struct {
    I2C_HandleTypeDef *i2c;
    uint8_t adresse; // déjà décalée <<1
    uint16_t dig_T1; int16_t dig_T2, dig_T3;
    uint16_t dig_P1; int16_t dig_P2, dig_P3, dig_P4, dig_P5,
                dig_P6, dig_P7, dig_P8, dig_P9;
    uint8_t dig_H1, dig_H3; int16_t dig_H2, dig_H4, dig_H5; int8_t dig_H6;
    int32_t t_fine; // intermédiaire de compensation
} bme280_t;

bool bme280_init(bme280_t *dev, I2C_HandleTypeDef *i2c, uint8_t addr7);
bool bme280_lire(bme280_t *dev, int32_t *temp_centiemes,
                uint32_t *press_pa, uint32_t *hum_milliemes);

#endif

```

Sorties en **point fixe** (centièmes de °C, Pa, millièmes de %) — **aucun float** dans le driver (module 01 : sur un petit micro on évite le flottant, et de toute façon les formules Bosch sont en entiers).

1:00-2:15 — Implémenter (datasheet à côté)

Étapes guidées :

1. **bme280_init** :
2. vérifier l'id (0xD0 → 0x60), sinon **return false** ;
3. lire les registres d'**étalonnage** (0x88..0xA1 pour T/P, 0xE1..0xE7 pour H) — le capteur stocke ses coefficients de correction en usine ;
4. configurer : **ctrl_hum** (0xF2), **ctrl_meas** (0xF4, oversampling ×1 + mode normal), **config** (0xF5).
5. **bme280_lire** :
6. lire **en rafale** les 8 octets 0xF7..0xFE (**HAL_I2C_Mem_Read** avec longueur 8) — une seule transaction pour des valeurs cohérentes entre elles ;
7. recomposer les mesures brutes 20 bits ;
8. appliquer les **formules de compensation** de la datasheet §4.2.3 (recopier ces formules est normal ; les comprendre est l'objectif).
9. **Gestion d'erreur** : chaque **HAL_I2C_...** qui échoue → **return false** . Le driver ne fait **jamais** de **printf** et ne bloque jamais plus que ses timeouts. C'est l'appelant qui décide quoi faire d'un échec.

Corrigé de référence de l'esprit (structure, point fixe, gestion d'erreur) : le driver BME280 est long ; le TP principal §Séance 2 en donne l'ossature. Si tu n'as pas le capteur : simule **bme280_lire** en renvoyant des valeurs dérivées d'un potentiomètre (l'architecture est le vrai objectif).

2:15-2:45 — Tester dans main

```
bme280_t capteur;
if (!bme280_init(&capteur, &hi2c1, 0x76))
    printf("BME280 absent !\r\n");

// dans la boucle (toutes les 1 s, non bloquant) :
int32_t t; uint32_t p, h;
if (bme280_lire(&capteur, &t, &p, &h))
    printf("T=%ld.%02ld C P=%lu Pa H=%lu.%03lu %%\r\n",
          t/100, t%100, p, h/1000, h%1000);
else
    printf("lecture BME280 echouee\r\n");
```

✓ **Point de contrôle** : valeurs plausibles ($T \approx 20-25$ °C, $P \approx 101325$ Pa, $H \approx 40-60$ %). Puis **débranche SDA en cours de route** : les `printf` doivent afficher « lecture echouee » sans que le programme gèle. Un driver qui gèle sur une déconnexion I2C est inutilisable en production.

2:45-3:00 — Commit + journal

`git commit -m "TP5 seance 2 : driver BME280 maison, point fixe, erreurs gerees"`. Journal : quelle formule de compensation t'a le plus surpris ? As-tu vérifié tes types (les coefficients signés/non signés du datasheet sont un piège) ?

Erreurs fréquentes

Symptôme	Cause	Remède
Scanner ne trouve rien	adresse non décalée, SDA/SCL inversés, pull-ups absentes	<code>a<<1</code> , vérifier câblage
id ≠ 0x60	mauvaise adresse, ou c'est un BMP280 (id 0x58)	vérifier la référence du module
Température ×256 ou absurde	coefficients d'étalonnage mal lus (signé/non signé)	respecter les types du datasheet
Valeurs figées	lecture pas en rafale (registres incohérents)	lire les 8 octets d'un coup
Gèle sur déconnexion	pas de test du retour HAL	<code>return false</code> sur échec, timeout court

Travail à la maison (45 min)

Écris un **test du driver compilable sur PC** : remplace les appels HAL par un « mock » qui renvoie des octets fixes (un tableau simulant les registres du capteur), et vérifie que ta compensation produit les valeurs attendues. C'est ainsi qu'on teste un driver sans le matériel (mentionné dans le sujet d'examen A « excellence »).

➡ Fiche suivante : [Séance 3 — ADC + DMA et console](#)